

Linux Class 10 - Practice Task

Practising sed line printing, line deleting and the memory commands

Submitted by: Shivakumar Muradi

Date: 02 October 2026

This is the practice I did after class 10. I worked inside a folder called practice10.

The file I practised on

I made a file called rk.txt with twelve lines. Every line is the name of an Indian dish and every line has the word masala in it. I did that on purpose, because the notes talk about counting occurrences of a word, and with one match on every line it is easy to count and check.

The twelve lines are masala dosa, masala idli, masala vada, masala chai, masala puri, masala papad, masala rice, masala curry, masala tea, masala roti, masala soup and masala bhaji.

Printing chosen lines with sed -n

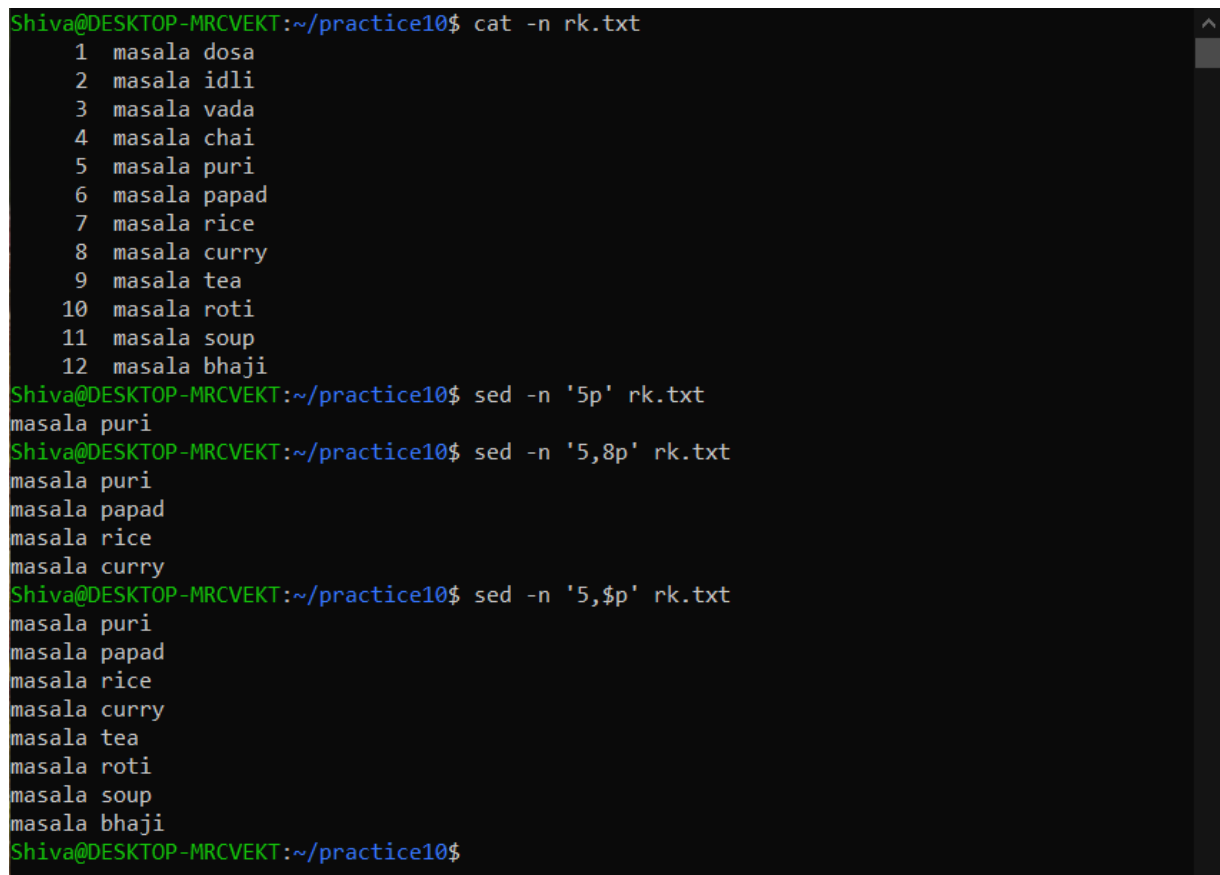
The -n tells sed not to print anything by itself, so the only lines that come out are the ones I ask for with p. I practised three forms of this.

```
sed -n '5p' rk.txt
```

```
sed -n '5,8p' rk.txt
```

```
sed -n '5,$p' rk.txt
```

I ran cat -n rk.txt first so that the real line numbers show in the screenshot. Then anybody can check that sed really picked the right lines.



```
Shiva@DESKTOP-MRCVEKT:~/practice10$ cat -n rk.txt
 1 masala dosa
 2 masala idli
 3 masala vada
 4 masala chai
 5 masala puri
 6 masala papad
 7 masala rice
 8 masala curry
 9 masala tea
10 masala roti
11 masala soup
12 masala bhaji
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed -n '5p' rk.txt
masala puri
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed -n '5,8p' rk.txt
masala puri
masala papad
masala rice
masala curry
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed -n '5,$p' rk.txt
masala puri
masala papad
masala rice
masala curry
masala tea
masala roti
masala soup
masala bhaji
Shiva@DESKTOP-MRCVEKT:~/practice10$
```

Screenshot 1: the numbered file, then line 5, then lines 5 to 8, then line 5 to the end.

Command	What it printed
<code>sed -n '5p' rk.txt</code>	Only line 5, which is masala puri.
<code>sed -n '5,8p' rk.txt</code>	Lines 5 to 8, so four lines together.
<code>sed -n '5,\$p' rk.txt</code>	Line 5 down to the last line. The \$ means the last line.

The thing I had to be careful about is the comma. A comma makes a range, so 5,8p is five through eight and not five and eight. If I wanted only line 5 and line 8 I would write `sed -n '5p;8p' rk.txt` with a semicolon instead.

Deleting lines with sed

Deleting works the same way as printing, only the letter changes from p to d. Here I do not use -n, because now I want sed to print the whole file and simply leave out the lines I named.

```
sed '4d' rk.txt
sed '4,8d' rk.txt
sed '4,$d' rk.txt
```

```
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed '4d' rk.txt
masala dosa
masala idli
masala vada
masala puri
masala papad
masala rice
masala curry
masala tea
masala roti
masala soup
masala bhaji
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed '4,8d' rk.txt
masala dosa
masala idli
masala vada
masala tea
masala roti
masala soup
masala bhaji
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed '4,$d' rk.txt
masala dosa
masala idli
masala vada
Shiva@DESKTOP-MRCVEKT:~/practice10$
```

Screenshot 2: one line deleted, then a range deleted, then everything from line 4 to the end deleted.

Command	What it printed
<code>sed '4d' rk.txt</code>	The file without line 4, so 11 lines.
<code>sed '4,8d' rk.txt</code>	The file without lines 4 to 8, so 7 lines.
<code>sed '4,\$d' rk.txt</code>	Only the first 3 lines, because 4 to the end are gone.

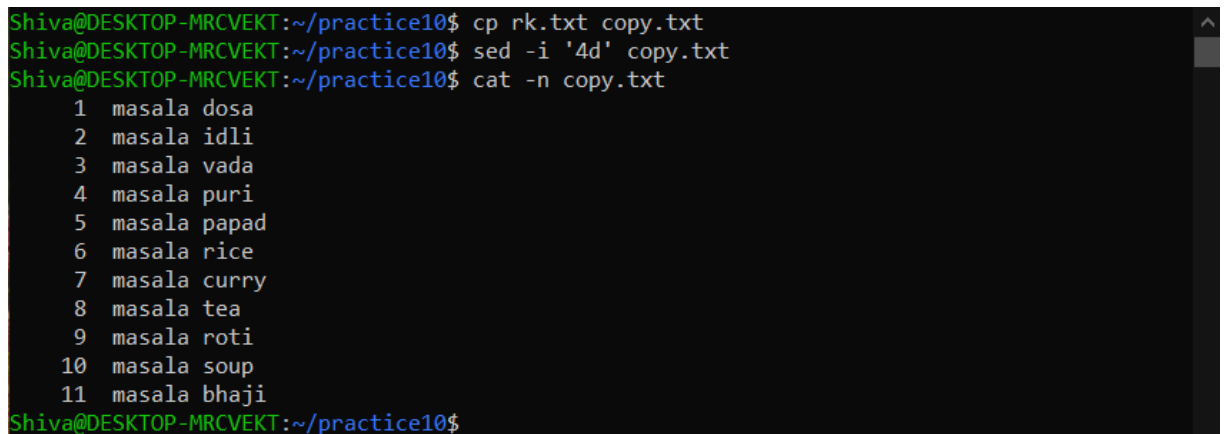
I counted the output each time to make sure. The first one gave 11 lines, the second gave 7 lines and the third gave only 3 lines. None of these changed rk.txt. They only changed what was printed on the screen, and the file still had its twelve lines afterwards.

Making the change stay, with -i

To actually change the file I need -i, which means in place. Because -i has no undo, I first made a copy and practised on the copy instead of on my original file.

```
cp rk.txt copy.txt
sed -i '4d' copy.txt
cat -n copy.txt
```

The sed -i line printed nothing at all, which confused me at first. Then I understood that this is correct, because the output went into the file instead of onto the screen. When I opened copy.txt with cat -n it had only 11 lines and masala chai was gone.



```
Shiva@DESKTOP-MRCVEKT:~/practice10$ cp rk.txt copy.txt
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed -i '4d' copy.txt
Shiva@DESKTOP-MRCVEKT:~/practice10$ cat -n copy.txt
 1 masala dosa
 2 masala idli
 3 masala vada
 4 masala puri
 5 masala papad
 6 masala rice
 7 masala curry
 8 masala tea
 9 masala roti
10 masala soup
11 masala bhaji
Shiva@DESKTOP-MRCVEKT:~/practice10$
```

Screenshot 3: after sed -i the copy has 11 lines and line 4 is missing.

Replacing the nth occurrence, and why -z is needed

The notes say that to replace the 3rd occurrence I write a number after the last slash:

```
sed 's/masala/SPICY/3' rk.txt
```

When I ran it, nothing changed. The whole file came out exactly as it was. That looked like a mistake but it is not. The number counts matches on each line separately, and my file has only one masala per line, so there is never a third match on any line.

The fix from the notes is the -z option. With -z, sed reads the whole file as one single piece instead of one line at a time, so the counting runs across the file:

```
sed -z 's/masala/SPICY/3' rk.txt
```

Now the third masala in the file changed, which is on line 3, and the rest stayed the same.

```
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed 's/masala/SPICY/3' rk.txt
masala dosa
masala idli
masala vada
masala chai
masala puri
masala papad
masala rice
masala curry
masala tea
masala roti
masala soup
masala bhaji
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed -z 's/masala/SPICY/3' rk.txt
masala dosa
masala idli
SPICY vada
masala chai
masala puri
masala papad
masala rice
masala curry
masala tea
masala roti
masala soup
masala bhaji
Shiva@DESKTOP-MRCVEKT:~/practice10$
```

Screenshot 4: without -z nothing changes, and with -z the 3rd masala becomes SPICY.

This was the most useful thing I learned in this class. The number after the slashes is not the line number. The line number goes in front, like `sed '3s/masala/SPICY/'`. The number at the end is the match number.

The 10th occurrence, and two patterns at once

To be sure I understood it, I asked for the 10th occurrence as well:

```
sed -z 's/masala/SPICY/10' rk.txt
```

Line 10, masala roti, became SPICY roti and nothing else moved. So the counting really is going straight down the file.

The notes also show how to do more than one replacement in a single command, by giving each one its own -e:

```
sed -e 's/masala/SPICY/g' -e 's/dosa/DOSA/g' rk.txt
```

Here the g means every match on the line, so all twelve masala words changed, and then the second instruction changed dosa as well. Both rules ran on the same line, one after the other.

```
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed -z 's/masala/SPICY/10' rk.txt
masala dosa
masala idli
masala vada
masala chai
masala puri
masala papad
masala rice
masala curry
masala tea
SPICY roti
masala soup
masala bhaji
Shiva@DESKTOP-MRCVEKT:~/practice10$ sed -e 's/masala/SPICY/g' -e 's/dosa/DOSA/g' rk.txt
SPICY DOSA
SPICY idli
SPICY vada
SPICY chai
SPICY puri
SPICY papad
SPICY rice
SPICY curry
SPICY tea
SPICY roti
SPICY soup
SPICY bhaji
Shiva@DESKTOP-MRCVEKT:~/practice10$
```

Screenshot 5: the 10th occurrence changed, and then two patterns changed in one command.

Checking disk space with du and df

The second half of the notes is about memory. The first two commands are du and df, and they are not the same thing.

- du means disk usage. It tells me how big my files and folders are.
- df means disk free. It tells me how much room is left on the drives.

I made a folder called sub with two copies of rk.txt inside it, so that du would have something to show.

```
du
```

```
du -h
```

```
du -h sub
```

```
df -h
```

Plain du printed numbers like 12 and 24. Those are blocks, not kilobytes, so they are hard to read. With -h the same sizes came out as 12K and 24K, which makes much more sense. The h stands for human readable.

```

Shiva@DESKTOP-MRCVEKT:~/practice10$ du
12    ./sub
24    .
Shiva@DESKTOP-MRCVEKT:~/practice10$ du -h
12K   ./sub
24K   .
Shiva@DESKTOP-MRCVEKT:~/practice10$ du -h sub
12K   sub
Shiva@DESKTOP-MRCVEKT:~/practice10$ df -h
Filesystem      Size  Used Avail Use% Mounted on
none            1.9G   0    1.9G   0% /usr/lib/modules/6.18.40.1-microsoft-standard-WSL2
none            1.9G  4.0K   1.9G   1% /mnt/wsl
drivers          238G   94G   145G  40% /usr/lib/wsl/drivers
/dev/sdd        1007G  1.8G   954G   1% /
none            1.9G   36K   1.9G   1% /mnt/wslg
none            1.9G   0    1.9G   0% /usr/lib/wsl/lib
rootfs          1.9G  3.3M   1.9G   1% /init
none            1.9G  784K   1.9G   1% /run
none            1.9G   0    1.9G   0% /run/lock
none            1.9G   0    1.9G   0% /run/shm
none            1.9G   80K   1.9G   1% /mnt/wslg/versions.txt
none            1.9G   80K   1.9G   1% /mnt/wslg/doc
C:\             238G   94G   145G  40% /mnt/c
D:\             467G  113M   467G   1% /mnt/d
E:\             465G  106G   359G  23% /mnt/e
tmpfs           1.9G   0    1.9G   0% /tmp
none            1.0M   0    1.0M   0% /run/credentials/systemd-journald.service
none            1.0M   0    1.0M   0% /run/credentials/systemd-resolved.service
snapfuse        45M   45M   0 100% /snap/snapd/28254
snapfuse        74M   74M   0 100% /snap/core22/2955
snapfuse        51M   51M   0 100% /snap/snapd/27738
snapfuse        4.5M  4.5M   0 100% /snap/tree/54
none            1.0M   0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           387M  12K   387M   1% /run/user/1000
Shiva@DESKTOP-MRCVEKT:~/practice10$

```

Screenshot 6: du, du -h, du -h on one folder, and then df -h for the whole system.

In the df -h output I could find the lines the notes mention. The line with /dev/sdd mounted on / is my Ubuntu root filesystem. The lines that say tmpfs are not real disks, they live in memory and disappear when the machine restarts. Because I am running Ubuntu inside WSL on Windows, my Windows drives also appear here as C:\ and D:\ mounted on /mnt/c and /mnt/d, which is why there are so many lines.

Memory, users and running processes

The last group of commands in the notes is about who is using the machine and how much memory is being used.

```
whoami
```

```
who
```

```
free -h
```

```
top -b -n 1 | head -8
```

whoami printed Shiva, which is my user name. who printed Shiva as well with the terminal pts/1 and the time I logged in, because I am the only one on this machine.

free -h showed that I have 3.8Gi of memory in total, about 522Mi used and 3.1Gi free at that moment.

```
Shiva@DESKTOP-MRCVEKT:~/practice10$ whoami
Shiva
Shiva@DESKTOP-MRCVEKT:~/practice10$ who
Shiva pts/1 2026-10-05 10:32
Shiva@DESKTOP-MRCVEKT:~/practice10$ free -h
              total        used        free      shared  buff/cache   available
Mem:           3.8Gi        522Mi        3.1Gi        4.3Mi        250Mi        3.3Gi
Swap:          1.0Gi           0B        1.0Gi
Shiva@DESKTOP-MRCVEKT:~/practice10$ top -b -n 1 | head -8
top - 10:58:45 up 1 day, 1:35, 1 user, load average: 0.16, 0.07, 0.12
Tasks: 31 total, 1 running, 30 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  2.2 sy,  0.0 ni, 97.8 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 3864.4 total, 3222.1 free, 524.0 used, 250.6 buff/cache
MiB Swap: 1024.0 total, 1024.0 free,  0.0 used. 3340.3 avail Mem

      PID USER      PR  NI   VIRT   RES    SHR S  %CPU  %MEM    TIME+  COMMAND
        1 root        20   0   24248  15384  11552 S   0.0   0.4   0:15.50 systemd
Shiva@DESKTOP-MRCVEKT:~/practice10$
```

Screenshot 7: whoami, who, free -h and the first 8 lines of top.

One honest note about top. The notes write it as top | head -8, but top is a live screen that keeps refreshing, so on my terminal that just hangs. I had to add -b for batch mode and -n 1 to take one reading, and then head -8 gave me the first 8 lines. Those 8 lines are the summary at the top of top: the uptime, the number of tasks, the CPU line, the memory line and the swap line.

Command	What it tells me
du	Size of this folder and the folders inside it, in blocks.
du -h	The same thing, but written as K, M or G so I can read it.
du -h sub	The size of one folder only.
df -h	Free and used space on every drive that is mounted.
free -h	How much RAM is used and how much is free.
top	A live list of running processes and the memory each one takes.
whoami	The name of the user I am logged in as.
who	All the users logged in right now.